Développement et déploiement d'applications intelligentes

Séance 3 — Étude de conception, projet reproductible, entraîner et comparer

Polycopié étudiant

Contenu	Page
L'essentiel du cours	1
Activités 1 et 2	3
Activité 3 : sections 4 et 5 de l'étude de conception	4
Bloc B : entraîner et comparer des modèles	5
Lab 2.1 : entraîner, évaluer et comparer des modèles	6
Pistes pour écrire le code	8
En cas de problème	9
Ticket de sortie	10

Un projet reproductible

Un projet est reproductible quand un autre que vous lance les commandes du README dans l'ordre et obtient le même résultat.

Cause d'échec	Pratique
Bibliothèque absente sur l'autre machine	Environnement virtuel et requirements.txt
Autre version d'une bibliothèque	Versions épinglées (==)
Chemin C:\Users\moi\data.csv	Chemins construits dans config.py
Cellules exécutées dans le désordre	Scripts lancés avec python -m
Résultat différent à chaque lancement	Graine fixe (SEED)
Personne ne sait si le résultat est bon	Tests avec pytest

Environnement virtuel ≠ machine virtuelle

Un **environnement virtuel** est un dossier `.venv` qui contient les bibliothèques Python du projet ; il se crée en une commande (`python -m venv .venv`) et s'active dans chaque nouveau terminal. Une **machine virtuelle** est un ordinateur entier simulé (VirtualBox, VMware) : lourde, et inutile dans ce cours.

Le projet à compléter

La structure du projet et huit tests sont fournis ; vous écrivez le code : charger les données, découper, choisir les modèles, évaluer, comparer. **Les tests sont l'énoncé** : au départ ils échouent, vous avez fini quand les huit passent. Tout se lance depuis la racine du projet : `python -m pytest`, `python -m monprojet.train`.

Réglage de config.py	Rôle
DATA_PATH	Votre fichier CSV, dans data/
TARGET	La colonne à prédire
TASK	« classification » ou « regression »
MAIN_METRIC	La métrique de votre section 5 : accuracy, recall, precision, f1 ; ou mae, rmse, r2
SPLIT, TIME_COLUMN	« random », ou « chronologique » avec la colonne de date pour une série temporelle
TEXT_COLUMN	Une colonne de texte libre (réclamations...), sinon None

Activité 1 — Faire tourner le projet d'un autre

En binôme, 7 minutes. Karim affirme : « ça marche chez moi, on obtient 14,2 % d'alertes ». Ouvrez **analyse.ipynb** (dans projet_a_reparer.zip) et notez tout ce qui empêche quelqu'un d'autre d'obtenir 14,2 %.

Activité 2 — Créer l'environnement et lancer le squelette

Chacun, 20 minutes.

1. Décompressez **projet_a_completer.zip** ; ouvrez un terminal dans le dossier projet_a_completer.
2. `python -m venv .venv`, puis `.venv\Scripts\Activate.ps1` (Windows) ou `source .venv/bin/activate` (macOS, Linux). L'invite affiche (.venv).
3. `pip install -r requirements.txt`
4. `python -m pytest` : attendu **7 failed, 1 passed**. C'est normal : rien n'est encore écrit.
5. Ouvrez data.py, models.py et train.py : lisez les commentaires TODO.
6. Essayez `python monprojet/train.py` : que se passe-t-il, et pourquoi ?

Votre réponse à la question 6 :

Quelles fonctions devrez-vous écrire, et dans quels fichiers ?

Activité 3 — Étude de conception : sections 4 et 5

En groupe, 25 minutes. Ces deux sections décident de ce que vous entraînerez cet après-midi.

Section 4 — Données

Le fichier exact : nom, lien, licence, nombre de lignes, colonnes utiles, présence de données personnelles. Pas encore de données ? Écrivez comment et quand vous les obtiendrez : c'est le premier risque de votre projet.

Section 5 — Métriques, baseline, critères

Trop vague	Utilisable
« Nous utiliserons l'accuracy. Le modèle doit être performant. »	« Baseline : toujours prédire « pas de panne » (accuracy 0,82, rappel 0). Métrique : le rappel, car une panne manquée arrête une ligne. Critère : rappel ≥ 0,75 et précision ≥ 0,50 sur le jeu de test (20 %, tenu à l'écart). »
« Nous prévoirons la consommation avec un réseau de neurones. »	« Baseline : la consommation de la même heure la semaine précédente. Métrique : MAE en kWh. Critère : MAE au moins 15 % sous la baseline, sur les deux derniers mois (découpage chronologique). »

Traduisez votre section 5 en réglages

Réglage	Votre valeur
DATA_PATH	
TARGET	
TASK	
MAIN_METRIC	
SPLIT (et TIME_COLUMN)	
TEXT_COLUMN	

Vérification par le groupe voisin

Pour chaque critère	Oui	Non
Une métrique est nommée		
Un nombre à atteindre		
Le jeu de données sur lequel on mesure		
La condition (seuil, période, comparaison à la baseline)		

Bloc B — Entraîner et comparer des modèles

Comparer honnêtement : quatre règles

- **Le même jeu de test** pour tous les modèles, jamais vu à l'entraînement, avec la même graine.
- **La baseline d'abord** : un modèle qui ne la bat pas n'apporte rien.
- **La métrique de votre section 5**, pas l'accuracy par défaut. Avec 12 % de pannes, « jamais de panne » a 88 % d'accuracy.
- **Le temps compte** : pour une série temporelle, entraîner sur le passé, tester sur le futur (SPLIT = « chronologique »).

Le jeu de test ne sert qu'à mesurer à la fin : si vous essayez vingt réglages et gardez celui qui a le meilleur score sur le test, ce score devient trop optimiste.

Les deux sprints suivent l'énoncé du Lab 2.1, page suivante : d'abord sur les données MaintAI, communes à tous, puis sur les données de votre projet.

Activité 4 — Le tableau des résultats (10 min)

Chaque groupe écrit au tableau : sujet, métrique, score de la baseline, score du meilleur modèle. Question pour la classe : le groupe au meilleur score a-t-il le meilleur modèle ?

Lab 2.1 — Entraîner, évaluer et comparer des modèles

Séance 3 · bloc B, sprints 1 et 2 (1 h 30) · thème 2.1 du syllabus

Objectifs

- Écrire soi-même le chargement des données, le découpage entraînement / test, les modèles candidats et leur évaluation.
- Comparer plusieurs modèles à une baseline, sur le même jeu de test, avec une métrique choisie à partir du coût des erreurs.
- Justifier le choix d'un modèle par écrit.
- Appliquer la même démarche aux données de votre projet.

Contexte : MaintAI

Une usine veut prédire les pannes de ses machines à partir de relevés de capteurs. Chaque matin, l'équipe de maintenance choisit les machines à inspecter. Une panne manquée arrête une ligne de production une journée ; une fausse alerte coûte une heure d'inspection. Le système doit donc détecter le plus de pannes possible, sans noyer l'équipe sous les fausses alertes.

C'est un problème de **classification binaire** : pour chaque relevé, prédire « panne » (1) ou « pas de panne » (0).

Les données

Le fichier `data/exemple_pannes.csv` contient 5 000 relevés, dont 17,6 % se terminent par une panne.

Colonne	Signification	Valeurs
temperature_c	Température du moteur (°C)	36 à 93, moyenne 65
vibration_mm_s	Vitesse de vibration (mm/s)	0 à 22, moyenne 3
load_pct	Charge, en % de la capacité nominale	20 à 100
runtime_hours	Heures de fonctionnement cumulées	0 à 30 000
failure	La cible : 1 = panne, 0 = pas de panne	0 ou 1

Ce qui est fourni, ce que vous écrivez

Fichier	Fourni	À écrire
config.py	Tous les réglages, déjà adaptés à MaintAI	—
data.py	feature_columns()	load_data(), split()
models.py	—	candidates()
train.py	build_preprocessing(), HIGHER_IS_BETTER, le plan de main()	evaluate(), main()
tests/	8 tests : ce qui est attendu	—

Partie 1 — Sur les données MaintAI (sprint 1 et début du sprint 2)

1. Écrivez `load_data()` et `split()` dans data.py. Découpage aléatoire, stratifié, 20 % en test, graine 42. Vérifiez : `python -m pytest` → **3 passed**.
2. Écrivez `candidates()` dans models.py : la baseline (la classe la plus fréquente) en premier, puis au moins trois modèles. → **5 passed**.
3. Écrivez `evaluate()` : accuracy, rappel, précision, F1 sur le jeu de test. → **6 passed**.
4. Écrivez `main()` en suivant le plan en commentaire. → **8 passed**.

5. `python -m monprojet.train` : le tableau de comparaison. **Vérification** : la baseline doit obtenir une accuracy de 0,824 et un rappel de 0. Si ce n'est pas le cas, votre découpage ou votre évaluation est à revoir.

Partie 2 — Sur les données de votre projet (fin du sprint 2)

1. Copiez votre fichier CSV dans `data/` et adaptez `config.py` : les réglages de votre activité 3 (TARGET, TASK, MAIN_METRIC, SPLIT…).
2. Relancez `python -m pytest` puis `python -m monprojet.train`. Votre code doit marcher sans autre modification : sinon, il dépendait de MaintAI quelque part (un nom de colonne écrit en dur, par exemple).
3. Pas encore de données ? Restez sur MaintAI et écrivez dans la section 4 comment et quand vous les obtiendrez.

Livrables

- Le code complété, avec les 8 tests verts (sur MaintAI, et sur vos données si vous les avez).
- `docs/choix-modele.md` rempli : le tableau des résultats, le modèle retenu, l'écart avec la baseline, le critère d'acceptation atteint ou non, les limites.
- Les sections 4 et 5 de `docs/etude-de-conception.md` réécrites.
- Le tout sera déposé sur le dépôt GitHub du groupe en séance 4.

Critères de réussite

Critère	Points	Ce qui est vérifié
Fonctionnement	6	Les 8 tests passent ; python -m monprojet.train produit le tableau et les fichiers de models/
Code	4	Aucun chemin ni nom de colonne écrit en dur hors de config.py ; le test n'est utilisé que pour évaluer
Tests	4	Le code passe les tests aussi sur vos données ; un test ajouté par le groupe (pour aller plus loin)
Documentation	6	choix-modele.md compare au moins trois modèles et la baseline, justifie le choix par la métrique de la section 5, discute les limites

Pour aller plus loin. Ajoutez un modèle avec class_weight="balanced" et comparez son rappel ; ou ajoutez un test qui vérifie que evaluate() renvoie un rappel de 0 pour la baseline sur MaintAI.

Pistes pour écrire le code

Les fonctions utiles, sans la solution. Chaque TODO du projet rappelle aussi ce qui est attendu.

À écrire	Ce qu'elle doit faire	Fonctions utiles
load_data()	Lire le CSV ; vérifier que la cible existe (sinon ValueError) ; retirer les lignes sans cible	pd.read_csv · df.columns · df.dropna(subset=[...])
split()	X = les colonnes de feature_columns(df), y = la cible. Aléatoire : test_size, graine, stratification pour une classification. Chronologique : trier par date, les plus récentes en test	train_test_split(X, y, test_size=..., random_state=..., stratify=...) · pd.to_datetime(...).argsort() · X.iloc[...]
candidates()	Un dictionnaire {nom: modèle} ; la baseline en premier ; au moins trois modèles	DummyClassifier(strategy="most_frequent") · DummyRegressor(strategy="mean") · LogisticRegression · LinearRegression · RandomForest… · HistGradientBoosting… · KNeighbors…
evaluate()	Prédire sur X_test ; renvoyer un dictionnaire de métriques arrondies (float)	model.predict · accuracy_score · recall_score · precision_score · f1_score (average, zero_division=0) · mean_absolute_error · mean_squared_error · np.sqrt · r2_score
main()	Pour chaque modèle : Pipeline([("preparation", build_preprocessing(X_train)), ("modele", modèle)]), fit sur l'entraînement, evaluate sur le test ; tableau ; meilleur modèle ; sauvegarde	pd.DataFrame(resultats).T · .idxmax() / .idxmin() · joblib.dump · json.dumps(..., ensure_ascii=False) · Path.write_text(..., encoding="utf-8")

Les erreurs les plus fréquentes

- Entraîner sur toutes les données au lieu de X_train : le test n'est plus un test.
- Évaluer sur le jeu d'entraînement : des scores trop beaux.
- Choisir le meilleur modèle avec idxmax pour une MAE : plus elle est petite, mieux c'est (HIGHER_IS_BETTER).
- « Object of type float32 is not JSON serializable » : convertir avec float(...).
- La cible parmi les variables d'entrée : utilisez feature_columns(df) pour X.

En cas de problème

Symptôme	Solution
python ouvre le Microsoft Store (Windows)	py -3.12 -m venv .venv
Activate.ps1 refusé	Set-ExecutionPolicy -Scope CurrentUser RemoteSigned, puis un nouveau terminal
ModuleNotFoundError: sklearn	Activez l'environnement dans ce terminal
ModuleNotFoundError: monprojet	Lancez depuis la racine du projet, avec python -m
Une seule colonne lue ; cible introuvable	Séparateur « ; » : dans data.py, pd.read_csv(path, sep=";")
Accents illisibles dans les noms de colonnes	pd.read_csv(path, encoding="latin-1")
Des nombres écrits « 12,5 » traités comme du texte	Les convertir dans data.py : df[col].str.replace(',', '.').astype(float)
Erreur sur une colonne de date	Indiquez-la dans TIME_COLUMN, ou listez FEATURES sans elle
ValueError dans le découpage stratifié	Une classe n'a qu'un exemple : regroupez ou retirez les classes rares
Le troisième test échoue	Le meilleur modèle ne bat pas la baseline : vérifiez la métrique, les variables, les données
Entraînement très lent	Échantillonnez pour la séance : df.sample(20000, random_state=42)

✂ - - -